Completed Lab Example

PostgreSQL + Express | Jason Owen | October 20, 2026

Branch: jason-owen/postgresql-lab
Submission: Pull request #57 — open for instructor review

This instructor worked example was prepared with AI assistance. The terminal commands, database queries, Node interactive POST, and Chrome browser checks were actually performed. Students should record and explain their own results.

Observed environment and results

Item	Observed value
Operating system	macOS 26.6.2, Apple Silicon
PostgreSQL	18.6 (Postgres.app)
Node.js / npm	v22.23.2 / 10.9.8
Database / login	csc425fa26 / csc425_student
Table	endpoint_examples.students
PostgreSQL / Express ports	5432 / 3001
Before POST	HTTP 200; five students
POST result	HTTP 201; Casey Diaz saved with ID 1007
After Express restart	HTTP 200; the same six students

Step 1 — Personal branch

```
git status
git switch main
git pull --ff-only origin main
git switch -c jason-owen/postgresql-lab
```

The working tree was clean and main was up to date. Main was missing the required demo files. I restored the previously prepared files and supplied PDF on this branch; the pull request includes them for students.

Step 2 — Existing installation inspected

PostgreSQL was already installed and accepting connections on port 5432. This Mac uses Postgres.app's binaries through a user login service. I verified that installation instead of initializing a second server.

Starting-condition difference: this is a completed run on an existing instructor database. The following page identifies the existing-database adjustment rather than claiming a fresh installation.

Steps 3–6 — Database and connection

Step 3 — Course login and database

Created the course login in psql and set its password interactively:

```
CREATE ROLE csc425_student LOGIN;
\password csc425_student
```

The database csc425fa26 already existed and belonged to csc425_demo. I preserved its records and owner, granting the new course login the access needed for this lab. I did not rerun CREATE DATABASE on an existing database.

For a fresh student installation, the guide uses:

```
CREATE DATABASE csc425fa26 OWNER csc425_student;
```

Connected using the guide's command and checked the identity:

```
\connect csc425fa26 csc425_student localhost 5432
SELECT current_database(), current_user;
```

current_database	current_user
csc425fa26	csc425_student

Step 4 — Seed records

Ran setup.sql in psql as csc425_student. It preserved the five fictional students. The SELECT check returned Avery Morgan, Jordan Bennett, Taylor Reed, Cameron Brooks, and Riley Parker.

Step 5 — Private local settings

```
DB_HOST=localhost
DB_PORT=5432
DB_NAME=csc425fa26
DB_USER=csc425_student
```

The actual password is intentionally omitted. It is stored in the ignored local .env file. The original instructor configuration was privately backed up. The command **git check-ignore server/.env** returned **server/.env**.

Step 6 — Dependencies and startup

```
npm ci --prefix server
npm run demo --prefix server
```

Express reported **PostgreSQL connected.** and the endpoint address **http://localhost:3001/api/students**.

npm ci succeeded. Its audit reported six advisories in the existing lockfile: one moderate, four high, and one critical. Dependency versions were not changed for this reproduction.

Steps 7–9 — Actual endpoint results

Step 7 — GET in Chrome

Opened **http://localhost:3001/api/students** in an isolated Chrome browser. The response was HTTP **200** with **five students**. The first record was:


```
{
  "studentID": "1001",
  "firstName": "Avery",
  "lastName": "Morgan",
  "major": "Computer Science"
}
```


Step 8 — POST from the Node prompt

Started **node** in a second terminal and ran the guide's fetch request once, with Casey Diaz and Computer Science in the JSON body. The status was **201**. The returned object contained:


```
{
  "message": "Student created.",
  "student": {
    "studentID": "1007",
    "firstName": "Casey",
    "lastName": "Diaz",
    "major": "Computer Science"
  }
}
```


Exited with **.exit** and checked GET in Chrome: **six students**, including Casey.

Casey's ID is 1007 because an earlier verification used the preceding sequence value. IDs need not be consecutive; a fresh database may assign 1006.

Step 9 — Restart and persistence

Stopped Express with **Ctrl+C**, ran **npm run demo --prefix server** again, and opened GET in Chrome. All six records matched the pre-restart response. A direct psql query as csc425_student confirmed:

ID	Name	Major
1001	Avery Morgan	Computer Science
1002	Jordan Bennett	Information Systems
1003	Taylor Reed	Business Administration
1004	Cameron Brooks	Mathematics
1005	Riley Parker	Computer Science
1007	Casey Diaz	Computer Science

Result: Casey remained because PostgreSQL stores the record independently of the Node process. The record was left in the local database for the classroom demonstration.

Explanation answers and submission

1. PostgreSQL and Express

PostgreSQL stores student records and executes SQL. Express handles HTTP requests in the Node application. A route asks PostgreSQL for data and decides what HTTP response to send.

2. What pool.query(...) does

It runs a SQL statement through a connection managed by the PostgreSQL pool and returns a promise for the result. result.rows contains the returned records. In POST, the values array supplies $1, $2, and $3 separately from the SQL text.

3. Why the route uses await

The database operation takes time. await pauses that async route handler until its query finishes so it can use the returned records. Other requests can still be processed while it waits.

4. From database record to JSON

The browser calls /api/students. Express selects the GET route and queries PostgreSQL. The pg driver exposes returned rows as JavaScript objects. res.json({ students: result.rows }) serializes them as JSON and sends the HTTP response.

For POST, express.json() first parses the body into req.body. The route validates and inserts the values. RETURNING provides the saved row, and res.status(201).json(...) sends it back.

5. Why Git branches do not change database rows

Git tracks repository files. PostgreSQL stores records in its own data directory. Switching branches changes tracked files, not database rows. A tracked setup script and its effects on the database are separate.

Step 10 — Commit, push, and pull request

```
git commit -m "Document PostgreSQL connection lab"
git push -u origin jason-owen/postgresql-lab
```

Lab submission commit: **9d86910**. The branch was published and pull request #57 opened with **base: main** and **compare: jason-owen/postgresql-lab**. It is left unmerged for instructor review. A follow-up commit includes this printable example and the confirmed submission link.

Final checks

[x] GET, POST, and restart/persistence verified locally.
[x] Four existing server tests passed; syntax and whitespace checks passed.
[x] Notes committed without credentials; branch and pull request published.

Full commands, results, and explanation answers: **server/endpointExamples/lab-notes-jason-owen.md**. This example reports local database and browser evidence, not a deployed service.